ДЕНЬ 1 (сегодня-завтра):

├── Трек A: Figma схема → PNG → PDF [ты + дизайнер]

├── Трек B: Форк OpenWebUI, настройка docker-compose базовой

│ структуры, подключение MWS GPT API [разработчик 1]

└── Трек B: Написание Smart Router (Pipe Function) +

TEXT_DEFAULT pipeline [разработчик 2]

ДЕНЬ 2:

├── Трек B: ASR pipeline (whisper-turbo)

├── Трек B: VLM pipeline (cotype-pro-vl-32b)

├── Трек B: Web Parser pipeline

└── Трек B: SearxNG интеграция + docker сервис

ДЕНЬ 3:

├── Трек B: Memory Manager (ChromaDB + bge-m3)

├── Трек B: RAG File pipeline

├── Трек B: Image Gen pipeline + fallback логика

└── Трек C: Заполнение шаблона фич (xlsx)

ДЕНЬ 4:

├── Трек B: Deep Research режим (доп. функционал)

├── Трек B: Генерация презентаций (доп. функционал)

├── Трек C: README + инструкция запуска

└── Трек D: Начало работы над презентацией (PDF, 15 слайдов)

ДЕНЬ 5 (финал):

├── Трек B: Финальное тестирование всех сценариев

├── Трек C: Финальный docker-compose проверка на чистой машине

├── Трек D: Запись demo-video (до 5 минут)

└── Трек D: Финализация презентации → сдача

ЧАСТЬ 2: Архитектура описанная словами — для разработчиков

🏛️ GPTHub — Техническое описание архитектуры

Общая идея

Мы берём OpenWebUI как готовый чат-интерфейс и дописываем к нему **три ключевых слоя**: умный роутер который определяет тип задачи, менеджер долгосрочной памяти, и набор пайплайнов для каждого типа запроса. Все AI-запросы идут через MWS GPT API который совместим с OpenAI API — это значит стандартные библиотеки (`openai` Python SDK) работают без изменений, просто меняем `base_url`.

Слой 1 — Интерфейс (трогаем минимально)

OpenWebUI уже даёт нам готовый чат с поддержкой файлов, изображений, Markdown, истории сообщений. Нам нужно добавить только:

- Выпадающий список моделей заполняется динамически через `GET /v1/models` от MWS GPT

- Кнопка микрофона уже есть — она использует Web Speech API браузера для записи и отдаёт текст в поле ввода

- Индикатор "какая модель сейчас активна" — небольшой badge рядом с полем ввода показывающий был ли выбор автоматическим или ручным

Слой 2 — Smart Router (главное что мы пишем)

Где живёт: OpenWebUI поддерживает механизм **Pipe Functions** — это Python-классы которые перехватывают запрос до того как он уходит к модели. Мы пишем один Pipe который является роутером.

Файл: `backend/apps/pipes/smart_router.py`

Как работает:

Когда пользователь отправляет сообщение, Pipe получает объект с полями: `messages` (история), `files` (вложения), `model` (выбранная модель или "auto").

Если модель выбрана вручную — Pipe пропускает запрос напрямую к указанной модели, ничего не делает.

Если модель "auto" — Pipe запускает функцию `detect_task_type()` которая смотрит на последнее сообщение пользователя и вложения:

```
detect_task_type() смотрит на:
1. files[].content_type — если audio/* → возвращает "ASR"
2. files[].content_type — если image/* → возвращает "VLM"
3. files[].content_type — если application/pdf или text/* → возвращает "RAG_FILE"
4. текст сообщения через regex → если есть https?:// → "WEB_PARSE"
5. текст через keyword matching → "найди"/"поищи"/"search" → "WEB_SEARCH"
6. текст через keyword matching → "нарисуй"/"сгенерируй"/"draw" → "IMAGE_GEN"
7. текст через keyword matching → "код"/"функция"/"def "/"bug" → "CODE"
8. len(всей истории в токенах) > 6000 → "LONG_CONTEXT"
9. иначе → "TEXT_DEFAULT"
```

По результату `detect_task_type()` роутер вызывает соответствующий пайплайн.

Слой 3 — Пайплайны (один на каждый тип задачи)

ASR Pipeline:

Получает аудиофайл → отправляет multipart/form-data POST на `https://api.gpt.mws.ru/v1/audio/transcriptions` с моделью `whisper-turbo` → получает текст транскрипта → подставляет транскрипт как текстовое сообщение → отправляет в TEXT_DEFAULT pipeline. Пользователь видит и транскрипт и ответ на него.

VLM Pipeline:

Получает изображение → кодирует в base64 → формирует messages в формате OpenAI vision: `[{"role": "user", "content": [{"type": "image_url", "image_url": {"url": "..."}}]}]` → отправляет на `/v1/chat/completions` с моделью `cotype-pro-vl-32b`.

RAG File Pipeline:

Получает файл → разбивает на чанки по 512 токенов с перекрытием 50 токенов → каждый чанк векторизует через `bge-m3` (`POST /v1/embeddings`) → сохраняет в ChromaDB с метаданными (session_id, filename, timestamp) → при запросе: векторизует вопрос пользователя → ищет top-5 похожих чанков в ChromaDB → добавляет их в системный промпт как контекст → отправляет в TEXT_DEFAULT pipeline.

Web Parse Pipeline:

Извлекает URL из текста сообщения через regex → делает GET запрос через `httpx` с таймаутом 10 секунд → парсит HTML через BeautifulSoup (берёт только `<p>`, `<h1>-<h3>`, `<article>` теги, убирает скрипты и стили) → обрезает до 3000 символов → добавляет в системный промпт: "Содержимое страницы {url}: {content}" → отправляет в TEXT_DEFAULT pipeline.

Web Search Pipeline:

Формирует поисковый запрос из сообщения пользователя → делает GET на `http://searxng:8888/search?q={query}&format=json` → берёт top-5 результатов (title, url, snippet) → форматирует как нумерованный список → добавляет в системный

промпт → отправляет в TEXT_DEFAULT pipeline. Ответ содержит источники которые OpenWebUI рендерит как кликабельные ссылки.

Image Gen Pipeline:

Формирует prompt из сообщения пользователя → пробует POST на `https://api.gpt.mws.ru/v1/images/generations` с моделью `sdxl-lightning-image` → если получает 200 — возвращает URL изображения который OpenWebUI рендерит inline → если получает 404 или 500 — переключается на локальный Stable Diffusion `http://stable-diffusion:7860/sdapi/v1/txt2img` → возвращает base64 изображения.

Code Pipeline:

Это TEXT_DEFAULT но с моделью `kodify-2.0` и системным промптом заточенным под код: "Ты эксперт-разработчик. Отвечай с примерами кода. Используй markdown code blocks с указанием языка."

TEXT_DEFAULT Pipeline:

Основной пайплайн. Перед отправкой к модели вызывает Memory Manager (см. ниже) → формирует messages со всей историей + системным промптом с извлечёнными фактами памяти → отправляет на `/v1/chat/completions` → после получения ответа снова вызывает Memory Manager для сохранения.

Слой 4 — Memory Manager

Где живёт: `backend/apps/memory/manager.py`

Два метода:

`retrieve_memory(user_id, query_text)`:

- Векторизует `query_text` через `bge-m3`
- Делает similarity search в ChromaDB в коллекции `user_memory_{user_id}`
- Возвращает top-3 факта с similarity > 0.75

- Эти факты добавляются в начало системного промпта: "Что ты знаешь о пользователе: {факты}"

`save_memory(user_id, message, response)`:

- Запускается асинхронно после получения ответа (не блокирует UI)

- Отправляет в LLM специальный запрос: "Из этого диалога извлеки конкретные факты о пользователе которые важно запомнить. Отвечай списком фактов или 'нет новых фактов'. Диалог: {message} → {response}"

- Если LLM вернул факты — векторизует каждый через `bge-m3` → сохраняет в ChromaDB с метаданными (user_id, timestamp, source_message_id)

- Дубликаты проверяются по similarity > 0.95 перед сохранением

Слой 5 — Инфраструктура (Docker)

Вся система поднимается одной командой `docker compose up -d` и содержит 4 контейнера:

openwebui — основное приложение, порт 3000 (UI) и 8080 (API). Собирается из нашего форка. Переменные окружения: `MWS_API_KEY`, `MWS_API_BASE_URL=https://api.gpt.mws.ru`.

chromadb — векторная база данных, порт 8000. Стандартный образ `chromadb/chroma`. Данные хранятся в volume `chroma_data`.

searxng — поисковый движок, порт 8888. Образ `searxng/searxng`. Конфиг через `searxng/settings.yml` где включены движки: Google, Bing, DuckDuckGo.

postgresql — база данных, порт 5432. Образ `postgres:15`. Volume `postgres_data`. Используется OpenWebUI для хранения пользователей и истории чатов.

Все контейнеры в одной Docker network `gpthub_net` и обращаются друг к другу по имени сервиса.

Точки расширения (дополнительный функционал)

Deep Research режим — команда `/research` запускает цепочку: SearxNG (5 запросов по подтемам) → Web Parser (парсит top-3 страницы по каждому запросу) → LLM суммаризирует каждый источник → финальный LLM синтезирует всё в структурированный отчёт с источниками.

Генерация презентаций — команда `/present` запускает: LLM генерирует структуру слайдов в JSON → Python скрипт использует `python-pptx` для создания .pptx файла → файл отдаётся пользователю как вложение для скачивания.

Командные пространства (Workspaces) — изолированные сессии на уровне БД: отдельная коллекция ChromaDB на workspace, общая история видна всем участникам, роли через таблицу `workspace_members` с полем `role` (admin/editor/viewer).